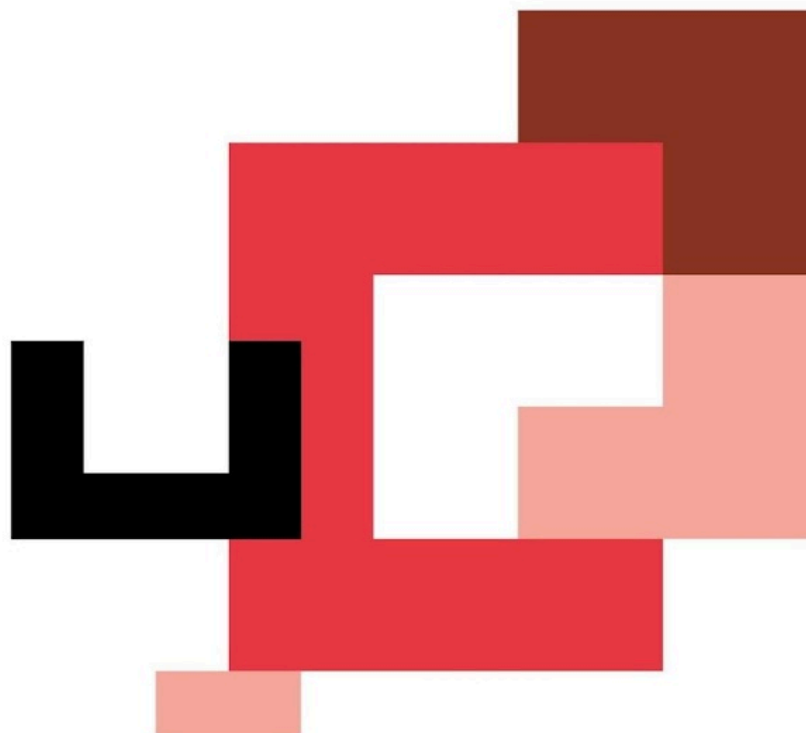




FULL STACK DEVELOPER + IA

Módulo 2 | Feature 2

Controladores, Servicios y
CRUD completo



Objetivo de la feature

En esta feature vamos a evolucionar la API creada en el sprint anterior incorporando una **arquitectura más profesional basada en capas**.

Hasta ahora las rutas gestionaban directamente la lógica. A partir de este sprint vamos a separar responsabilidades para que el proyecto sea más escalable y mantenible.

Qué aprenderás:

- Cómo **organizar un backend profesional** separando rutas, controladores y servicios.
- Cómo implementar el **CRUD completo** para un recurso (**products**).
- Cómo gestionar **validaciones básicas y errores HTTP correctamente**.
- Cómo mantener un **formato de respuesta estándar en JSON**.

El objetivo final es entender **cómo se estructura una API real antes de introducir bases de datos o autenticación**.

Estructura del proyecto

None

```
src/  
  
├─ controllers/  
|   └─ products.controller.js  
  
├─ services/  
|   └─ products.service.js  
  
├─ routes/  
|   └─ index.routes.js  
|       └─ products.routes.js  
  
├─ db/  
|   └─ products.js  
  
├─ app.js  
  
└─ server.js
```

Explicación de cada capa

Routes

Define los endpoints de la API.

Ejemplo:

None

GET /api/products

POST /api/products

Las rutas **no deben contener lógica de negocio**.

Controllers

Se encargan de:

- Recibir la request
 - Validar datos de entrada
 - Llamar a los servicios
 - Devolver la respuesta HTTP
-

Services

Aquí vive la **lógica de negocio**.

Los servicios:

- Manipulan datos
 - Aplican reglas
 - Devuelven información a los controladores
-

Endpoints

Método	Ruta	Descripción
GET	/health	Estado del servidor
GET	/api/products	Listar productos
GET	/api/products/:id	Obtener producto
POST	/api/products	Crear producto
PUT	/api/products/:id	Actualizar producto
DELETE	/api/products/:id	Eliminar producto

Body POST / PUT

```
JSON
{
  "name": "string (obligatorio)",
  "price": "number (obligatorio, >= 0)",
  "description": "string (opcional)",
  "stock": "number (opcional)",
}
```



```
"imageUrl": "string (opcional)"  
}
```

Qué hay que hacer (paso a paso)

1) Crear estructura de carpetas (algunas ya las tienes)

Dentro de `src` crea:

```
None  
controllers  
  
services  
  
routes  
  
db
```

2) Crear servicios

En `services/products.service.js` debes crear funciones como:

- `getAllProducts()`
- `getProductById(id)`
- `createProduct(data)`
- `updateProduct(id, data)`
- `deleteProduct(id)`

Estos métodos trabajarán con los datos en memoria.

3) Crear controladores

En:

None

```
controllers/products.controller.js
```

Debes:

- Validar body de POST y PUT
 - Manejar errores
 - Llamar a los servicios
 - Devolver respuestas estándar JSON
-

5) Conectar rutas

En:

None

```
routes/products.routes.js
```

Conecta cada endpoint con su controlador.

Ejemplo conceptual:

None

```
GET /api/products
```

```
→ controller.getAllProducts
```

6) Mantener formato estándar de respuesta

Todas las respuestas deben seguir este formato.

Éxito

JSON

```
{  
  
  "ok": true,
```

```
"data": {}  
  
}
```

Error

```
JSON  
  
{  
  
  "ok": false,  
  
  "error": {  
  
    "message": "Error message"  
  
  }  
  
}
```

Uso de la IA en este sprint

La IA debe ayudarte a pensar el código, no a copiarlo sin entenderlo.

Puedes usarla para:

- Revisar arquitectura Controller / Service
- Detectar casos que faltan
- Revisar validaciones
- Crear datos para poder trabajar con ellos partiendo de algo así (si no lo hubieras creado)

```
JSON  
  
{  
  
  id: '1',  
  
  name: 'Camiseta Básica',  
  
}
```

```
description: 'Camiseta de algodón 100%',  
price: 19.99,  
stock: 50,  
imageUrl: 'https://placeholder.co/300x300?text=Camiseta',  
createdAt: '2024-01-01T00:00:00.000Z',  
}
```

Prompts útiles (después de haberlo hecho todo)

Arquitectura

None

Estoy construyendo una API en Express con arquitectura
routes → controllers → services.

Explícame qué responsabilidades debe tener cada capa
y qué errores debo evitar.

Revisión CRUD

None

Tengo estos endpoints en Express:

GET /api/products

GET /api/products/:id

POST /api/products


```
PUT /api/products/:id
```

```
DELETE /api/products/:id
```

¿Qué validaciones y casos de error debería cubrir?

Debug

None

Tengo este error en Express al crear un producto.

Te paso el controlador y el servicio.

Dime las tres causas más probables.

Pistas

1. Los controladores no deben tener lógica de negocio

Los controladores solo coordinan.

La lógica vive en **services**.

2. Validaciones básicas

Para crear un producto debes validar:

- `name` obligatorio
- `price` obligatorio
- `price >= 0`

Si falla → **400**

3. Uso correcto de HTTP

Caso	Código
Producto creado	201
Datos inválidos	400
Producto no encontrado	404

Importante que tienen que tener en cuenta

1. Todo es JSON

No se renderizan vistas.

No se usa HTML.

2. Formato estándar

Siempre:

```
None  
{ ok:true, data }
```

o

```
None  
{ ok:false, error }
```

3. IDs numéricos

Los IDs llegarán como **string** desde la URL:

```
None  
req.params.id
```

Normalmente debes convertirlos a número.

4. Separación real de capas

No hagas esto:

```
None  
routes → acceso directo a db
```

Siempre:

```
None  
routes → controller → service
```

Checks rápidos (autoevaluación)

El sprint está correcto si:

- `npm start` levanta el servidor
 - `GET /health` funciona
 - `GET /api/products` devuelve array
 - `GET /api/products/1` devuelve producto
 - `GET /api/products/999` devuelve **404**
 - `POST /api/products` crea producto
 - `PUT /api/products/:id` actualiza
 - `DELETE /api/products/:id` elimina
-

Ejemplos cURL

Listar productos

Shell

```
curl http://localhost:3000/api/products
```

Obtener producto

Shell

```
curl http://localhost:3000/api/products/1
```

Crear producto

Shell

```
curl -X POST http://localhost:3000/api/products \
-H "Content-Type: application/json" \
-d '{"name":"Gorra Deportiva","price":14.99}'
```

Actualizar producto

Shell

```
curl -X PUT http://localhost:3000/api/products/1 \
-H "Content-Type: application/json" \
-d '{"name":"Camiseta Premium","price":24.99}'
```

Eliminar producto

Shell

```
curl -X DELETE http://localhost:3000/api/products/1
```

Validación fallida

Shell

```
curl -X POST http://localhost:3000/api/products \  
-H "Content-Type: application/json" \  
-d '{"description":"Sin nombre ni precio"}'
```

Notas

- Los datos siguen siendo **temporales (memoria)**.
- Mantener siempre la estructura **routes** → **controllers** → **services**.